

03 — CDC: Change Data Capture with PostgreSQL

Table of Contents

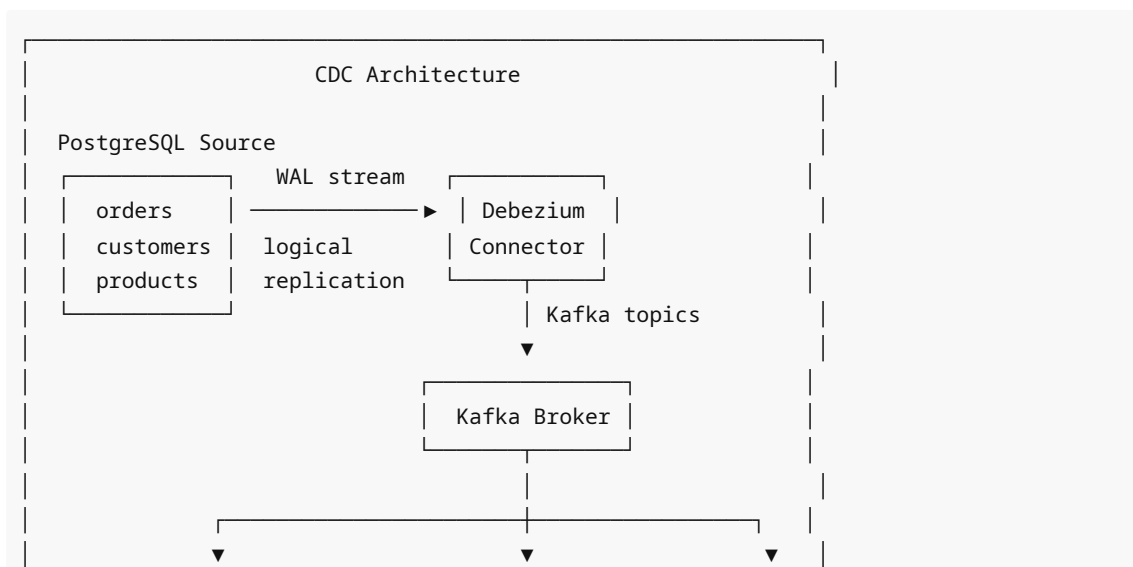
1. [Learning Objectives](#)
 2. [What is CDC?](#)
 3. [CDC Patterns Overview](#)
 4. [PostgreSQL WAL and Logical Decoding](#)
 5. [Logical Replication Setup](#)
 6. [Debezium + PostgreSQL](#)
 7. [WAL2JSON Output Plugin](#)
 8. [Polling-Based CDC](#)
 9. [Trigger-Based CDC](#)
 10. [Common Mistakes](#)
 11. [Best Practices](#)
 12. [Performance Considerations](#)
 13. [Interview Questions & Answers](#)
 14. [Exercises & Solutions](#)
 15. [Real-World Scenarios](#)
 16. [Cross-References](#)
-

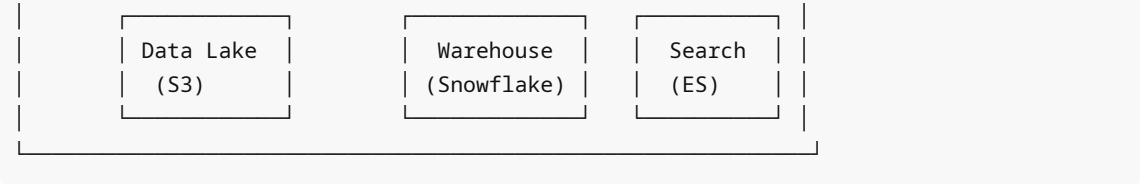
Learning Objectives

- Explain CDC concepts and differentiate approaches
 - Configure PostgreSQL for logical decoding / WAL-based CDC
 - Set up Debezium connector with a complete configuration file
 - Implement trigger-based and polling-based CDC as alternatives
 - Identify pitfalls such as replication slot lag and WAL retention
-

What is CDC?

Change Data Capture tracks every INSERT, UPDATE, and DELETE made to a source database and streams those changes to downstream systems in near real-time.





CDC Patterns Overview

Pattern	Mechanism	Latency	Overhead	Completeness
WAL / Logical Rep	PostgreSQL write-ahead log	seconds	Low	Full (I/U/D)
Debezium	WAL + Kafka Connect	seconds	Medium	Full (I/U/D)
Polling / Watermark	updated_at column	minutes	Low	No deletes
Trigger-based	DB triggers to audit table	seconds	High	Full (I/U/D)
Log shipping	Archived WAL files	minutes	Very low	Full

PostgreSQL WAL and Logical Decoding

Prerequisites: postgresql.conf

```
# Enable logical replication (requires restart)
wal_level = logical

# How many replication slots to allow
max_replication_slots = 10

# How many WAL sender processes
max_wal_senders = 10

# Retain WAL for up to 1GB per slot (prevent disk bloat)
max_slot_wal_keep_size = 1GB
```

pg_hba.conf — allow replication connections

```
# Allow Debezium / connector user to replicate
host replication debezium_user 10.0.0.0/8 scram-sha-256
```

Create replication user

```
CREATE ROLE debezium_user WITH
  LOGIN
  REPLICATION
  PASSWORD 'strong_password_here';

-- Grant SELECT on tables to capture
```

```
GRANT SELECT ON ALL TABLES IN SCHEMA public TO debezium_user;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT ON TABLES TO debezium_user;

-- Grant usage on publication (PostgreSQL 10+)
GRANT CREATE ON DATABASE mydb TO debezium_user;
```

Create a Publication

```
-- Publish specific tables
CREATE PUBLICATION cdc_pub
    FOR TABLE orders, customers, products;

-- Publish entire schema (PostgreSQL 15+)
CREATE PUBLICATION cdc_pub_all
    FOR TABLES IN SCHEMA public;

-- Check what is published
SELECT * FROM pg_publication_tables WHERE pubname = 'cdc_pub';
```

Create Logical Replication Slot

```
-- pgoutput plugin (built-in, preferred)
SELECT pg_create_logical_replication_slot('debezium_slot', 'pgoutput');

-- wal2json plugin (alternative, rich JSON output)
SELECT pg_create_logical_replication_slot('json_slot', 'wal2json');

-- View existing slots
SELECT slot_name, plugin, active, restart_lsn, confirmed_flush_lsn,
    pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS
    retained_wal
FROM pg_replication_slots;
```

Read changes manually (pgoutput)

```
-- Peek without consuming
SELECT * FROM pg_logical_slot_peek_binary_changes(
    'debezium_slot', NULL, NULL,
    'proto_version', '1',
    'publication_names', 'cdc_pub'
);

-- Consume changes (advances slot LSN)
SELECT * FROM pg_logical_slot_get_binary_changes(
    'debezium_slot', NULL, 10,
    'proto_version', '1',
```

```
'publication_names', 'cdc_pub'
);
```

Debezium + PostgreSQL

Debezium Connector Configuration (complete example)

```
{
  "name": "postgres-cdc-connector",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
    "database.hostname": "db.internal.example.com",
    "database.port": "5432",
    "database.user": "debezium_user",
    "database.password": "strong_password_here",
    "database.dbname": "production",
    "database.server.name": "prod_pg",

    "plugin.name": "pgoutput",
    "slot.name": "debezium_slot",
    "publication.name": "cdc_pub",

    "table.include.list": "public.orders,public.customers,public.products",

    "topic.prefix": "cdc",
    "topic.creation.enable": "true",
    "topic.creation.default.replication.factor": "3",
    "topic.creation.default.partitions": "6",

    "snapshot.mode": "initial",
    "snapshot.isolation.mode": "repeatable_read",

    "decimal.handling.mode": "double",
    "time.precision.mode": "adaptive",
    "interval.handling.mode": "numeric",

    "heartbeat.interval.ms": "5000",
    "heartbeat.action.query": "UPDATE debezium_heartbeat SET last_heartbeat =
NOW()",

    "tombstones.on.delete": "true",

    "transforms": "unwrap",
    "transforms.unwrap.type": "io.debezium.transforms.ExtractNewRecordState",
    "transforms.unwrap.drop.tombstones": "false",
    "transforms.unwrap.delete.handling.mode": "rewrite",
    "transforms.unwrap.add.fields": "op,source.ts_ms",
    "transforms.unwrap.add.headers": "db,table",

    "errors.tolerance": "all",
```

```

    "errors.deadletterqueue.topic.name": "dlq.cdc.errors",
    "errors.deadletterqueue.context.headers.enable": "true",

    "poll.interval.ms": "500",
    "max.batch.size": "2048",
    "max.queue.size": "16384",

    "schema.history.internal.kafka.bootstrap.servers": "kafka:9092",
    "schema.history.internal.kafka.topic": "schema-changes.production"
  }
}

```

Deploy via Kafka Connect REST API

```

# Register connector
curl -X POST http://kafka-connect:8083/connectors \
  -H "Content-Type: application/json" \
  -d @debezium-pg-connector.json

# Check status
curl http://kafka-connect:8083/connectors/postgres-cdc-connector/status | jq .

# List connectors
curl http://kafka-connect:8083/connectors

# Restart on error
curl -X POST http://kafka-connect:8083/connectors/postgres-cdc-connector/restart

```

Debezium Change Event Structure

```

{
  "before": {
    "order_id": 12345,
    "status": "pending",
    "amount": 99.99
  },
  "after": {
    "order_id": 12345,
    "status": "shipped",
    "amount": 99.99
  },
  "op": "u",
  "ts_ms": 1705305600000,
  "source": {
    "db": "production",
    "schema": "public",
    "table": "orders",
    "lsn": 12345678,
    "txId": 9876
  }
}

```

```
}  
}
```

Heartbeat table (prevents WAL retention bloat)

```
-- Create heartbeat table (must be in publication)  
CREATE TABLE debezium_heartbeat (  
    id          INT          PRIMARY KEY DEFAULT 1,  
    last_heartbeat TIMESTAMPZ  
);  
INSERT INTO debezium_heartbeat VALUES (1, NOW());  
  
-- Add to publication  
ALTER PUBLICATION cdc_pub ADD TABLE debezium_heartbeat;
```

WAL2JSON Output Plugin

```
-- Install (requires Debian/Ubuntu package: postgresql-16-wal2json)  
SELECT pg_create_logical_replication_slot('wal2json_slot', 'wal2json');  
  
-- Read JSON changes  
SELECT data::json  
FROM pg_logical_slot_get_changes(  
    'wal2json_slot', NULL, 10,  
    'format-version', '2',  
    'include-timestamp', 'true',  
    'include-pk', 'true',  
    'include-schemas', 'true'  
);
```

Sample wal2json output:

```
{  
  "action": "U",  
  "schema": "public",  
  "table": "orders",  
  "pk": [{ "name": "order_id", "type": "int8", "value": 12345 }],  
  "columns": [  
    { "name": "status", "type": "text", "value": "shipped" },  
    { "name": "updated_at", "type": "timestampz", "value": "2024-01-15T10:30:00Z" }  
  ],  
  "identity": [  
    { "name": "status", "type": "text", "value": "pending" }  
  ],  
  "timestamp": "2024-01-15T10:30:00.123456Z"  
}
```

Polling-Based CDC

Simple approach when WAL is unavailable (e.g., RDS without logical replication enabled):

```
-- Ensure updated_at column exists
ALTER TABLE orders ADD COLUMN IF NOT EXISTS updated_at TIMESTAMPTZ DEFAULT NOW();
CREATE INDEX idx_orders_updated_at ON orders (updated_at);

-- Trigger to auto-update
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_orders_updated_at
BEFORE UPDATE ON orders
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

```
# Python polling CDC script
import psycopg2
import datetime

def poll_cdc(conn, last_watermark):
    with conn.cursor() as cur:
        cur.execute("""
            SELECT order_id, customer_id, amount, status, updated_at
            FROM orders
            WHERE updated_at > %s
            ORDER BY updated_at
            """, (last_watermark,))
        rows = cur.fetchall()

    if rows:
        process_changes(rows)
        new_watermark = rows[-1][4] # updated_at of last row
        save_watermark(new_watermark)
        return new_watermark
    return last_watermark
```

Limitation: Cannot detect DELETE operations without a `deleted_at` soft-delete column.

Trigger-Based CDC

```
-- Audit table to capture all changes
CREATE TABLE audit_log (
```

```

audit_id    BIGSERIAL PRIMARY KEY,
table_name  TEXT      NOT NULL,
operation   TEXT      NOT NULL,  -- INSERT, UPDATE, DELETE
old_data    JSONB,
new_data    JSONB,
changed_by  TEXT      DEFAULT current_user,
changed_at  TIMESTAMPTZ DEFAULT NOW(),
transaction_id BIGINT  DEFAULT txid_current()
);

-- Generic audit trigger function
CREATE OR REPLACE FUNCTION audit_trigger_func()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        INSERT INTO audit_log (table_name, operation, old_data)
        VALUES (TG_TABLE_NAME, 'DELETE', row_to_json(OLD)::JSONB);
        RETURN OLD;
    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO audit_log (table_name, operation, old_data, new_data)
        VALUES (TG_TABLE_NAME, 'UPDATE', row_to_json(OLD)::JSONB,
row_to_json(NEW)::JSONB);
        RETURN NEW;
    ELSIF TG_OP = 'INSERT' THEN
        INSERT INTO audit_log (table_name, operation, new_data)
        VALUES (TG_TABLE_NAME, 'INSERT', row_to_json(NEW)::JSONB);
        RETURN NEW;
    END IF;
END;
$$;

-- Attach to orders table
CREATE TRIGGER trg_orders_audit
AFTER INSERT OR UPDATE OR DELETE ON orders
FOR EACH ROW EXECUTE FUNCTION audit_trigger_func();

```

Common Mistakes

Mistake	Impact	Fix
Not setting wal_level = logical	Logical replication fails	Set in postgresql.conf and restart
Inactive replication slot	WAL accumulates, disk fills	Monitor pg_replication_slots.active, drop unused slots
No max_slot_wal_keep_size	Disk OOM from WAL retention	Set max_slot_wal_keep_size = 1GB
Missing heartbeat	Slot never advances on idle tables	Add heartbeat table to publication

Replicating large objects / TOAST	Events missing large column values	Set <code>REPLICA IDENTITY FULL</code> for tables with TOAST
Snapshot mode = always	Full re-snapshot on restart	Use <code>snapshot.mode = never</code> after initial load

Best Practices

1. **Monitor replication slot lag** — alert when `retained_wal > 5GB`.
2. **Always use a heartbeat table** — keeps slot LSN advancing even when source is quiet.
3. **Set `REPLICA IDENTITY FULL`** for UPDATE/DELETE to include old values:

```
ALTER TABLE orders REPLICA IDENTITY FULL;
```

4. **Use `pgoutput`** (built-in) over third-party plugins when possible.
5. **Dead-letter queue** for failed Debezium events — never silently drop.
6. **Separate CDC user** with minimal permissions (REPLICATION + SELECT only).
7. **Test failover** — verify slot survives primary failover in HA setups.

Performance Considerations

```
-- Monitor WAL retention by slot
SELECT
    slot_name,
    active,
    pg_size_pretty(
        pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)
    ) AS retained_wal,
    pg_size_pretty(
        pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn)
    ) AS pending_flush
FROM pg_replication_slots
ORDER BY pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) DESC;

-- Identify hot tables for CDC (high write frequency)
SELECT schemaname, relname, n_tup_ins + n_tup_upd + n_tup_del AS writes_per_min
FROM pg_stat_user_tables
ORDER BY writes_per_min DESC
LIMIT 20;
```

Interview Questions & Answers

Q1: What is CDC and why is it useful?

CDC captures every INSERT, UPDATE, and DELETE from a source database in near real-time, enabling downstream systems (warehouses, search indexes, caches) to stay synchronized without full table scans.

Q2: What is `wal_level = logical` and why is it needed for CDC?

PostgreSQL's WAL normally records physical block changes (minimal level). `logical` level adds row-level change information needed to reconstruct which rows changed, enabling logical replication and CDC.

Q3: What is a replication slot and what risk does it carry?

A replication slot tracks the WAL position a subscriber has consumed. PostgreSQL retains all WAL since the slot's `restart_lsn`. If a consumer falls behind or dies, WAL accumulates and can fill the disk.

Q4: How does Debezium handle schema changes?

Debezium writes schema history to a dedicated Kafka topic (`schema.history.internal.kafka.topic`). On restart, it replays schema history to reconstruct the correct schema at each LSN position.

Q5: What is `REPLICA IDENTITY FULL` and when do you need it?

By default, PostgreSQL only includes primary key columns in the `before` image of UPDATE/DELETE events. `REPLICA IDENTITY FULL` includes all columns, needed when consumers need old values for deduplication or auditing.

Q6: How do you handle CDC for a table without a primary key?

Set `REPLICA IDENTITY FULL` to include all columns as the unique identifier, or add a surrogate primary key. Without any identity, PostgreSQL cannot reliably identify changed rows.

Q7: Explain the difference between snapshot mode `initial` and `never` in Debezium.

`initial` performs a full table snapshot on first run, then switches to streaming. `never` skips snapshot (useful if the target is already populated) and streams only new changes.

Q8: How do you monitor CDC pipeline health in production?

Monitor `pg_replication_slots.confirmed_flush_lsn` lag, Kafka consumer group lag, Debezium connector status via Kafka Connect REST API, and heartbeat event latency.

Q9: What is a heartbeat table and why does Debezium need one?

A heartbeat table receives periodic updates even when source tables are idle. This advances the replication slot's LSN, preventing WAL accumulation and giving Debezium a regular event to emit heartbeat messages.

Q10: How would you implement CDC for soft deletes?

Add a `deleted_at TIMESTAMPTZ` column. CDC captures the UPDATE that sets `deleted_at`. Downstream consumers treat `deleted_at IS NOT NULL` as a logical delete. This is simpler but requires application-level enforcement.

Q11: What happens if a Debezium connector is paused for 24 hours?

WAL accumulates since `restart_lsn`. If `max_slot_wal_keep_size` is set, PostgreSQL may invalidate the slot once WAL exceeds the limit, requiring a full re-snapshot.

Exercises & Solutions

Exercise 1: Set up logical replication slot

```
-- 1. Verify wal_level
SHOW wal_level; -- must be 'logical'
```

```

-- 2. Create publication for orders
CREATE PUBLICATION orders_cdc FOR TABLE orders;

-- 3. Create replication slot
SELECT pg_create_logical_replication_slot('orders_slot', 'pgoutput');

-- 4. Monitor retained WAL
SELECT slot_name,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn))
FROM pg_replication_slots;

```

Exercise 2: Trigger-based audit trail

```

-- Apply audit trigger to customers table
CREATE TRIGGER trg_customers_audit
  AFTER INSERT OR UPDATE OR DELETE ON customers
  FOR EACH ROW EXECUTE FUNCTION audit_trigger_func();

-- Test
INSERT INTO customers (id, name) VALUES (999, 'Test User');
UPDATE customers SET name = 'Updated User' WHERE id = 999;
DELETE FROM customers WHERE id = 999;

-- Check audit log
SELECT * FROM audit_log WHERE table_name = 'customers' ORDER BY changed_at;

```

Real-World Scenarios

Scenario: E-commerce order status sync Orders table receives ~5,000 updates/second at peak. Debezium streams changes to Kafka. Two consumers: (1) Elasticsearch for real-time search, (2) Snowflake for analytics. Heartbeat interval = 5 seconds. REPLICA IDENTITY FULL on orders to capture status change from old value.

Scenario: GDPR right-to-erasure CDC captures all user data changes. When a delete request arrives, the `deleted_at` column is set (soft delete). A nightly job removes rows with `deleted_at < NOW() - 30 days` and publishes tombstone events to Kafka so downstream caches and search indexes can purge.

Cross-References

- `01_etl_with_postgresql.md` — logical decoding as ETL source
- `06_incremental_loading.md` — watermark patterns as polling CDC alternative
- `08_postgresql_and_airflow.md` — orchestrating CDC consumers
- `../17_PostgreSQL_For_Backend_Engineers/08_scaling_patterns.md` — CQRS and event sourcing with CDC